

Fundamentals of System Security (COMSM0122)

Individual Coursework: Tiered Infrastructure Security (60 Marks)

Submission Deadline: Friday 21st November 2025 @16.00

Overview

This coursework explores the design, deployment, and evaluation of a tiered infrastructure using Infrastructure as Code provisioning platform (Terraform) and Configuration Management (Ansible). The system provisions multiple virtual machines and configures them into distinct tiers: load balancer (HAProxy), web servers (Node.js), application, and administration. The assessment is structured into two phases followed by three tasks: (1) applying software design patterns and principles such as layered architecture and separation of concerns; (2) evaluating infrastructure health through routing algorithms, resource scaling, and VM replication; and (3) enforcing confidentiality and integrity using access control matrices and formal security models (Bell-LaPadula and Biba). Through practical experimentation, simulation, and policy design, students gain hands-on experience in building scalable, maintainable, and secure infrastructures.

Intended Learning Outcomes

- Apply design patterns to infrastructure architecture.
- Demonstrate separation of concerns in provisioning and configuration.
- Evaluate system health under varying loads and configurations.
- Implement access control and security models to enforce confidentiality and integrity.

Resources:

- <https://registry.terraform.io/providers/bpg/proxmox/latest/docs>
- <https://docs.ansible.com>

Phase 1: Provision Multi-Tier Infrastructure with Terraform

Download the file **tiered_infra.zip** from Blackboard. This file contains Terraform configuration which provisions virtual machines and HAProxy load balancer infrastructure on a Proxmox Virtual Environment cluster. Before using this Terraform setup, ensure you have:

1. **Terraform** installed (version 1.0+)
2. **Proxmox Virtual Environment** access with:
 - API endpoint URL
 - Valid API token or credentials
3. **SSH Keys**: Public and private key pair for VM access
4. **Proxmox Provider Plugin**: Configured via Terraform (**bpg/proxmox v0.86.0**)
5. **Ansible** installed (for phase 2)

Study the project structure. There are two modules in the project. First is the VMs Module (**modules/vms/**), which provisions virtual machines on the Proxmox cluster by cloning VMs from a Ubuntu cloud image template, carrying out cloud-init configurations and injecting SSH keys via credentials path. The second is the HAProxy module (**modules/haproxy/**), which creates the infrastructure of a HAProxy load balancer, including its provisioning, cloud-init setup and network configuration. In both cases, the setup first does a lookup of existing VMs to check a name match, offering you the control to create VMs and HAProxy only when they do not exist. You can toggle this control to true or false by changing the values of **force_recreate_haproxy** and **force_recreate_vms** in the root **variables.tf** file.

Also note the following tiered approach applied in the design.

1. Load Balancer Tier (HAProxy) – acts as the traffic director, distributing incoming requests across multiple web servers. The purpose is to ensure high availability (if one web server fails, traffic goes to another) and better performance through load sharing. The identity tags are **tier_loadbalancer** and **role_haproxy**.
2. Web Tier – handles client-facing requests (HTTP/API) forwarded by HAProxy. This tier runs the Node.js API servers that respond to /api calls. The identity tags are **tier_web** and **role_frontend**.
3. App Tier – This is the business logic layer and it's responsible for processing data, applying rules and interacting with databases or external services. In a full setup, the web tier would call the app tier for deeper logic. The identity tags are **tier_app** and **role_backend**.
4. Admin Tier – This is responsible for management and monitoring. It provides tools for system administration, dashboards, or control panels. This tier is used by developers/ops teams to manage the environment, not by end users. The identity tags are **tier_admin** and **role_management**.

You are expected to edit placeholders in **variables.tf** located in the root module or provide values via **terraform.tfvars**. Similarly, replace **<your-uni-id>** in haproxy and vms modules **clone.tf** files.

Follow Terraform commands to initialise, plan and apply the infrastructure description. A successful execution will generate an output showing the IP addresses of the HA proxy and the VMs created, as illustrated in the snapshot below. This IP will be required as input in phase 2..

```
haproxy_created = {  
  "created" = true  
  "ip" = "192.168.0.117"  
  "message" = "HAProxy VM successfully cloned."  
  "name" = "ubuntu-haproxy-clone"  
  "tags" = tolist([  
    "env_dev",  
    "role_haproxy",  
  ])
```

```

        "tier_loadbalancer",
    ])
}

vms_created = {
    "admin-01" = {
        "created" = true
        "ip" = "192.168.0.116"
        ...
    }
    .
    .
}

```

Phase 2: Infrastructure configuration and orchestration

To orchestrate the infrastructure deployment pipeline, this phase adds a layer of **Ansible** on top of the infrastructure provisioned in phase 1. The focus is on implementing a playbook that configures HAProxy and deploys a simple **Node.js API server** on the web tier. The implementation of this layer can be found in the **ansible-playbook** folder of the downloaded zip file.

The playbook provisions HAProxy as a load balancer and deploys Node.js API servers on the web tier, wiring them together so the environment responds to API calls through a single entry point. The **inventory (host.ini/host.yml)** defines hosts for the HAProxy tier and Web tier using the IPs from Terraform and specifying the directory path to private keys. Remember to replace these with your actual IP and directory path. There are two roles defined in the playbook. The **haproxy role** installs HAProxy, applies a **Jinja2** config template, and restarts the service. The second is the **web role**, which installs **Node.js** and **npm**, as well as deploys a simple Express API (**/api** endpoint), sets up a **systemd** service, and ensures it runs on port 3000. The **Playbook (site.yml)**: then runs the **haproxy** role on the load balancer VM and the **web** role on the web tier VMs. The overall flow is as follows:

- a) HAProxy is installed and configured to forward requests to the web tier.
- b) Node.js API servers are deployed on **web-01** and **web-02**.
- c) HAProxy balances incoming traffic across both web servers.
- d) Clients can access the API via **http://<haproxy_ip>/api**.

The outcome is a working load-balanced Node.js API service where HAProxy distributes requests between multiple web servers to ensure scalability and resilience. Remember to replace the **<xx-ip-address>** and **<your-directory-path>** placeholders in **host.ini/host.yml**.

Run the following commands from **ansible-playbook** directory to execute the playbook:

- (1) To inspect inventory:
`ansible-inventory -i inventories/dev/hosts.yml --list`
- (2) To test connectivity (simple ping)

```
ansible -i inventories/dev/hosts.yml all -m ping -u ubuntu --
<your_dir_path>/.ssh/id_rsa
```

(3) Syntax check a playbook

```
ansible-playbook -i inventories/dev/hosts.yml playbooks/site.yml --
syntax-check
```

(4) Run the playbook

```
ansible-playbook -i inventories/dev/hosts.yml playbooks/site.yml -v
```

The figure below is a sample of a successfully running playbook

```
PLAY RECAP *****
ubuntu-haproxy-clone : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
web-01               : ok=7    changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
web-02               : ok=7    changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Task 1: Infrastructure Design Patterns (10 Marks)

(a) The infrastructure designed in phases 1 and 2 provisions multiple virtual machines using Terraform and then configures them with Ansible. The architecture separates responsibilities into distinct tiers: load balancer (HAProxy), web tier (Node.js API servers), app tier, and admin tier.

In no more than 150 words, discuss which software design pattern this architecture most closely resembles, and how it improves scalability and maintainability in the system. (5 Marks)

(b) The above project uses Terraform to provision virtual machines and Ansible to configure them into specific tiers (HAProxy load balancer, Node.js web servers, app tier, and admin tier).

Discuss how this setup demonstrates the “Separation of Concerns” as a design principle. In your answer, explain why keeping infrastructure provisioning and configuration management distinct improves reliability and team collaboration. (5 Marks)

Task 2: Infrastructure Health (25 Marks)

You will explore the impact of different HAProxy routing algorithms on the health of the web tier using your Ansible inventory setup. Follow the approach highlighted below that combines configuration templating, deployment, and traffic simulation.

(a) Use Jinja2 templates to define multiple **haproxy.cfg.j2** config files, each with a distinct routing algorithm. Also, write the corresponding version of the playbook that allows switching between each template. (5 Marks)

(b) Simulate traffic of the current infrastructure setup using either: (a) **curl** in a loop from multiple clients; (b) **ab** (ApacheBench); or (c) **wrk** or **hey** for load testing –

these are command-line tools used for HTTP load testing and benchmarking web servers. Analyse the traffic behaviour with changing test load by enabling HAProxy stats interface and including the code block below in your HAProxy .cfg file. Remember to restart the HAProxy after the configuration update. You can access behaviour via browser: `http://<ip_address_of_haproxy>:8080/stats`.

```
listen stats
  bind *:8080
  mode http
  stats enable
  stats uri /stats
```

Starting with a test load of 1000, generate 15 data points by increasing each load by a factor of 1000. Observe and record the connection distribution, server load, and health status for each test load. Repeat for each routing algorithm by refactoring the necessary file in your playbook to point to the next version of the algorithm derived from (a), redeploy, and rerun traffic simulations.

Plot a graph to illustrate your results. In no more than 150 words, report on the impact of the choice of routing algorithm on the health of the web tier. **(5 Marks)**

- (c) The current infrastructure is based on a configuration that uses a single core and sets the memory of a web tier VM clone to 512MB and the disk space to 10GB. Assume that the system has been deployed to production, and due to high demand, the current configuration is failing to satisfy its high-availability contract. Your task is to explore which alternatives to increase memory, processing capability, or disk space are more efficient. You will explore this challenge by repeating Task(2b) based on a refactored code with the following configurations:
- I. Fixed CPU and hard disk, and memory of 512, 1024, 1536 and 2048 MB respectively.
 - II. Fixed CPU and memory, and hard disk sizes of 10,20, 30 and 40 GB respectively.
 - III. Fixed memory and hard drive, and CPU of 1 and 2 cores, respectively.

Plot a graph based on the results from (I), (II) and (III). In no more than 150 words, discuss your configuration recommendation to the engineering team. **(10 Marks)**

- (d) The engineering team is considering an alternative option of leaving the current configuration setting or provisioning more VMs at the web tier. Generate two separate refactorings of the original design. The first, by adding one more VM called `<your-uni-id>-web-03`, and the second by adding two more VMs called `<your-uni-id>-web-03` and `<your-uni-id>-web-04`, respectively. For each refactoring, repeat Task(2b).

Plot a graph to illustrate your results. In no more than 150 words, report on the impact of VM replication and the choice of routing algorithm on the health of the web tier. **(5 Marks)**

Task 3: Access Control Matrix (25 Marks)

The infrastructure resulting from implementing phases 1 and 2 consists of (1) A HAProxy tier (<haproxy-ip-address>), which assumes the role of the Load balancer, and a single entry point; (2) the Web tier (web-01 and web-02), providing frontend Node.js REST API functionality; (3) the Admin tier (admin-01), used for management and configuration; and (4) the App tier (app-01) for hosting backend services. Using this three-tier architecture, you will design and apply an access control system to enforce confidentiality and integrity controls.

- (a) Construct an access control matrix that specifies which tier can initiate connections to which tier and what operations are permitted. These include the following:

Subject (Tier)	Resource (Tier)	Permitted Operations	Rationale
HAProxy	Web		
Web	Admin		
Web	App		
Admin	HAProxy		
App	Web		

Complete the matrix with permissions from a set containing READ, WRITE, EXECUTE, ADMIN, FORWARD, DENY, and ALL. Justify each entry based on the infrastructure architecture. **(5 Marks)**

- (b) The Bell-LaPadula model enforces confidentiality based using the **simple security property** - a subject at level L cannot read data at level L' where $L' > L$ (no read-up), and the **star property** - a subject at level L cannot write to level L' where $L' < L$ (no write-down).

The following security levels are assigned to tiers:

- **Level 0 (Public):** HAProxy (public entry point)
- **Level 1 (Confidential):** Web tier (processes user data)
- **Level 2 (Secret):** Admin tier (configuration, credentials)
- **Level 3 (Top Secret):** App tier (sensitive backend data)

- Which information flows violate the simple security property (no read-up)?
- Which information flows violate the Star Property (no write-down)?
- Enforce the rules from (i) and (ii) by refactoring the initial setup code and defining the firewall rule using bpg/proxmox provider.

(7 Marks)

- (c) The Biba model enforces integrity based on the **simple integrity property** - a subject at integrity level I cannot read data at level I' where $I' < I$ (no read-down), and **star property** - a subject at integrity level I cannot write to level I' where $I' > I$ (no write-up).

The following integrity levels are assigned to tiers (higher = more trustworthy):

- **Level 3 (High):** Admin tier (trusted configuration source)
 - **Level 2 (Medium):** App tier (processes validated inputs)
 - **Level 1 (Low):** Web tier (handles untrusted user input)
 - **Level 0 (Lowest):** HAProxy (external, untrusted entry point)
- (i) Which information flows violate the Simple Integrity Property (no read-down)? explain the risk.
- (ii) Which information flows violate the Star Property (no write-up)? Why is this important.
- (iii) Define bpg/proxmox provider firewall rules to prevent the web tier from corrupting admin-tier configuration files? Discuss your answer.

(7 Marks)

- (d) Bell-LaPadula and Biba often conflict. For example, Bell-LaPadula allows web to admin writes (Star Property), but Biba forbids it (no write-up).

- (i) Identify 3 conflicts between the two models in your infrastructure.
- (ii) For each conflict, propose a policy that satisfies both models (e.g., via data sanitisation, role-based access, audit logging).

(6 Marks)

Deliverables

Submission should be made electronically as a folder containing all deliverables via Blackboard for group coursework. Your folder should be named coursework-[your studentid]. The folder should contain the following:

1. A separate zip file containing the code for each task solution that required refactoring the setup code.
2. A PDF document containing your answers to Tasks 1 - 3.

Assessment

Submission is due by Friday, 21st November 2025 @16.00. As per the Code of Assessment policy, information on late submission penalties can be found at: <https://www.bristol.ac.uk/students/support/academic-advice/outcomes/mark-signals/>.

All submissions will be checked against our plagiarism monitoring system.

Marking Rubric

Task & Criterion	Excellent (Full Marks)	Good	Fair	Poor
Task 1: Design Patterns (10)				
(a) Pattern (5)	Identifies layered/tiered pattern; clear ≤150 words; strong scalability & maintainability link.	Relevant pattern; partial explanation .	Mentions pattern, vague benefits.	Incorrect/no link.
(b) Separation of Concerns (5)	Clear Terraform vs Ansible roles; strong link to reliability & collaboration.	Mentions separation; partial benefits.	Mentions tools, weak principle link.	No clear understanding.
Task 2: Infrastructure Health (25)				
(a) Templates & Playbook (5)	Multiple correct templates; switching playbook works.	Mostly correct; minor errors.	Limited variation/incomplete.	Minimal/incorrect.
(b) Traffic Simulation (5)	15 data points; graphs; ≤150-word clear analysis.	Attempted; fewer points/partial analysis.	Minimal simulation; weak analysis.	No attempt.
(c) Resource Scaling (10)	Both experiments complete; graphs; clear ≤150-word recommendation.	Mostly complete; partial justification .	Partial experiments; weak/missing recommendation.	No meaningful work.
(d) VM Replication (5)	Both refactorings; simulations; graphs; clear ≤150-word analysis.	One refactoring; partial analysis.	Minimal attempt.	No work.

Task 3: Access Control & Models (25)				
(a) Access Matrix (5)	Complete, correct permissions; strong rationale.	Mostly correct; partial rationale.	Incomplete/weak rationale.	Incorrect/missi ng.
(b) Bell-LaPadul a (7)	Correct violations; firewall refactoring; strong justification.	Mostly correct; partial enforceme nt.	Limited identification; weak rules.	Incorrect/missi ng.
(c) Biba Integrity (7)	Correct violations; risks explained; firewall rules; strong discussion.	Mostly correct; partial explanation .	Limited identification; weak rules.	Incorrect/missi ng.
(d) Conflict Resolution (6)	3 conflicts; realistic policies satisfying both models.	Conflicts identified; partial solutions.	Minimal conflicts; weak solutions.	No attempt.